

NAME: Ayaan Shaikh

Mobile no : 8055733707

Internship : Data Analytics using Python

Submission date: 27/05/2026

Task 101 : Perform Hyperparameter Tuning on Dataset.

Objective

To optimize model performance by selecting the best combination of parameters (e.g., learning rate, depth, estimators) using techniques like Grid Search or Random Search.

```
# 101. Perform Hyperparameter Tuning on Dataset
from sklearn.ensemble import RandomForestClassifier
import pandas as pd
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import train_test_split
df=pd.read_csv("User_Data_Association.csv")
X=df[['Age','EstimatedSalary']]
y=df['Purchased']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2)
param_grid={
    'n_estimators':[50,100],
    'max_depth':[3,5,10]
}
model=RandomForestClassifier()
grid=GridSearchCV(model,param_grid,cv=5)
grid.fit(X_train,y_train)
print("Best Parameters:", grid.best_params_)
best_model=grid.best_estimator_

Best Parameters: {'max_depth': 3, 'n_estimators': 100}
```

I loaded the dataset and selected **Age** and **EstimatedSalary** as input features, while **Purchased** was taken as the target variable.

Then I split the data into training and testing sets to evaluate how well the model performs on unseen data. After that, I defined a set of hyperparameters (n_estimators and max_depth) to tune the **Random Forest Classifier**.

Using **GridSearchCV with 5-fold cross-validation**, I tested all possible combinations of these parameters to find the best-performing model.

Once the training was complete, I printed the best parameters and correctly extracted the optimized model using `best_estimator_`, which represents the final tuned model trained with the best parameter combination. This process helps me achieve better accuracy, reduce overfitting, and improve the overall performance of my machine learning model.

Task 102 : Use ARIMA Model for Time Series Forecasting.

Objective

The main objective of using the **ARIMA (AutoRegressive Integrated Moving Average) model** is to analyze time-based data and predict future values by capturing patterns such as trends, seasonality, and past dependencies in the data.

```
# 102. Use ARIMA Model for Time Series Forecasting
from statsmodels.tsa.arima.model import ARIMA
ts=df['EstimatedSalary']
model=ARIMA(ts,order=(1,1,1))
model_fit=model.fit()
forecast=model_fit.forecast(steps=3)
print("Forecast : ",forecast)

Forecast :    400    46435.762697
401    45214.299806
402    45357.266989
Name: predicted_mean, dtype: float64
```

I selected the **EstimatedSalary** column as a time series and applied the **ARIMA model** with parameters (1,1,1), where 1 represents the autoregressive part, 1 represents differencing to make the data stationary, and 1 represents the moving average part. I then trained the model using the `fit()` method, which helps the model learn patterns and dependencies from past values. After training, I used the `forecast(steps=3)` function to predict the next three future values of the salary data. Finally, I printed the forecasted results, which represent the model's prediction of future trends based on historical data.

Task 103 : Create visuals for Variance & Standard Deviation.

Objective :

The main objective of using **Bayesian Inference** is to create a probabilistic model that makes predictions by combining prior knowledge (previous beliefs) with new data. It helps in continuously updating the model as more data becomes available using **Bayes' Theorem**.

```
# 103. Build Model using Bayesian Inference
from sklearn.naive_bayes import GaussianNB
model=GaussianNB()
model.fit(X_train,y_train)
print("NB Accuracy : ",model.score(X_test,y_test))

NB Accuracy : 0.875
```

I used the **Gaussian Naive Bayes algorithm**, which is based on **Bayesian Inference**, to build a classification model. I created the model using `GaussianNB()` and trained it on the training dataset using `model.fit(X_train, y_train)`, allowing the model to learn the relationship between input features and the target variable.

After training, I evaluated the model's performance on the test dataset using `model.score(X_test, y_test)`, which calculates the accuracy.

Finally, I printed the accuracy to understand how well the model predicts unseen data.

Task 104 : Study Overfitting and Underfitting in Machine Learning .

Objective

The objective is to understand how a machine learning model behaves when it is too simple (underfitting) or too complex (overfitting). It helps ensure the model learns the right patterns from data and performs well on both training and unseen data.

```
# 104.Study Overfitting and Underfitting in Machine Learning
train_acc=model.score(X_train,y_train)
print("Training Accuracy : ",train_acc)
test_acc=model.score(X_test,y_test)
print("Testing Accuracy : ",test_acc)

Training Accuracy :    0.909375
Testing Accuracy :    0.8625
```

I calculated both the **training accuracy** and **testing accuracy** of my machine learning model to study overfitting and underfitting. I used the training data to check how well the model learned the patterns, which gave me an accuracy of about **90.94%**, and then evaluated it on unseen test data, where it achieved around **86.25%** accuracy. By comparing these two values, I analyzed the model's performance and observed that the difference is small, indicating that the model is not overfitting or underfitting but is generalizing well on new data.

Task 105: Apply Cross Validation and Grid Search Techniques.

Objective

The objective is to improve model performance by selecting the best hyperparameters using **Grid Search** and to ensure model reliability using **Cross Validation**. These techniques help in reducing overfitting and achieving better generalization.

```
# 105.Apply Cross Validation and Grid Search Techniques
from sklearn.model_selection import cross_val_score
score=cross_val_score(model,X_train,y_train,cv=3)
print("CV Accuracy : ",score.mean())

CV Accuracy : 0.8906130606007171
```

I applied **Cross Validation** using the `cross_val_score` function to evaluate my model more reliably. Instead of testing the model on a single split, I used **3-fold cross-validation (cv=3)**, where the training data is divided into three parts, and the model is trained and tested multiple times on different combinations of these parts. This helped me reduce bias and get a more accurate estimate of the model's performance. Finally, I calculated the **mean accuracy** of all folds using `score.mean()` to understand the overall performance of my model.

Task 106 : Use Ensemble Methods (Bagging, Boosting) .

Objective

The objective of using **Ensemble Methods (Bagging and Boosting)** is to improve model accuracy and stability by combining multiple models. These techniques reduce errors like variance and bias, leading to better predictions.

```
# 106. Use Ensemble Methods (Bagging, Boosting)
from sklearn.ensemble import GradientBoostingClassifier
rf=RandomForestClassifier()
gb=GradientBoostingClassifier()
rf.fit(X_train,y_train)
gb.fit(X_train,y_train)
print("RF : ",rf.score(X_test,y_test))
print("Gb : ",gb.score(X_test,y_test))

RF :  0.9375
Gb :  0.925
```

I applied **Ensemble Methods** by using both **Random Forest (Bagging)** and **Gradient Boosting (Boosting)** algorithms to compare their performance. I trained both models on the training dataset using `fit()` and then evaluated them on the test dataset using `score()`. Random Forest works by combining multiple decision trees trained on different subsets of data to reduce variance, while Gradient Boosting builds models sequentially, where each new model corrects the errors of the previous one to reduce bias. By printing their accuracies, I was able to compare which ensemble method performs better on my dataset and select the most effective model.

Task 107 : Use Hierarchical Clustering for Cluster Analysis .

Objective

The objective of using **Hierarchical Clustering** is to group similar data points into clusters without pre-defining the number of clusters. It helps in understanding data structure and relationships using a tree-like hierarchy.

```
# 107. Use Hierarchical Clustering for Cluster Analysis
from sklearn.cluster import AgglomerativeClustering
X=df[['Age','EstimatedSalary']]
model=AgglomerativeClustering(n_clusters=3)
labels=model.fit_predict(X)
df['Cluster']=labels
df.head()
```

Output:

	User ID	Gender	Age	EstimatedSalary	Purchased	Cluster
0	15624510	Male	19	19000	0	1
1	15810944	Male	35	20000	0	1
2	15668575	Female	26	43000	0	1
3	15603246	Female	27	57000	0	2
4	15804002	Male	19	76000	0	2

I used Hierarchical Clustering (Agglomerative Clustering) to perform cluster analysis on my dataset based on features like Age and Estimated Salary. I selected the relevant columns and applied the model with `n_clusters=3`, which means I grouped the data into three clusters. Using `fit_predict()`, I generated cluster labels for each data point and then added these labels as a new column in my dataset. This helped me understand how users are grouped based on similar characteristics and identify patterns within the data.

Task 108 : Split Train/Test Dataset.

Objective

The objective of splitting a dataset into **training and testing sets** is to evaluate the model's performance on unseen data. It helps ensure that the model generalizes well and avoids overfitting.

```
# 108. Split Train/Test Dataset
from sklearn.model_selection import train_test_split
df=pd.read_csv("User_Data_Association.csv")
X=df[['Age','EstimatedSalary']]
y=df['Purchased']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
print(X_train)
```

	Age	EstimatedSalary
3	27	57000
18	46	28000
202	39	134000
250	44	39000
274	57	26000
..	...	...
71	24	27000
106	26	35000
270	43	133000
348	39	77000
102	32	86000

[320 rows x 2 columns]

I used the `train_test_split()` function from `sklearn` to divide my dataset into training and testing sets. I selected Age and EstimatedSalary as input features and Purchased as the target variable. I split the data with `test_size=0.2`, which means 80% of the data is used for training and 20% for testing, and I set `random_state=42` to ensure consistent and reproducible results. After splitting, I printed `X_train` to view the training feature data that will be used to train my model.

Task 109 : Perform Feature Engineering .

Objective

The objective of **Feature Engineering** is to improve model performance by creating, transforming, or selecting the most relevant features. It helps the model understand patterns in data more effectively.

```
# 109. Perform Feature Engineering
from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
print("Successfully Performed Feature Engineering")

Successfully Performed Feature Engineering
```

I performed Feature Engineering by applying Standardization using the StandardScaler. I fitted the scaler on the training data using fit_transform() to learn the mean and standard deviation, and then applied the same transformation to the test data using transform() to maintain consistency. This step ensures that features like Age and Estimated Salary are scaled to a similar range, which helps improve the performance of machine learning models, especially those sensitive to feature magnitude. Finally, I printed a message confirming that feature engineering was successfully completed.

Task 110 : Build Prediction Model based on Age, Education, and Inputs .

Objective

The objective is to build a **prediction model** using features like Age, Education, and other inputs to predict the target variable. It helps in making data-driven decisions based on user characteristics.


```

# 110. Build Prediction Model based on Age, Education, and Inputs
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
df=pd.read_csv("User_Data_Association.csv")
data = {
    'Age': [25, 30, 35, 40, 28, 32, 45, 50],
    'Education': ['UG', 'PG', 'UG', 'PhD', 'PG', 'UG', 'PhD', 'PG'],
    'Salary': [30000, 50000, 40000, 80000, 45000, 38000, 90000, 60000],
    'Purchased': [0, 1, 0, 1, 1, 0, 1, 1]
}

df = pd.DataFrame(data)
df
le=LabelEncoder()
df['Education']=le.fit_transform(df['Education'])
X=df[['Age','Education','Salary']]
y=df['Purchased']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
model=LogisticRegression()
model.fit(X_train,y_train)
new=[[27,2,40000]]
print("Prediction : ",model.predict(new))

Prediction : [0]

```

I built a prediction model using Logistic Regression based on features like Age, Education, and Salary. Instead of using the CSV dataset, I created my own sample dataset using a dictionary and converted it into a DataFrame. Since Education is a categorical variable (UG, PG, PhD), I used LabelEncoder to convert it into numerical values so that the model can process it. Then, I selected input features and the target variable (Purchased), and split the data into training and testing sets using train_test_split(). After that, I trained the Logistic Regression model using the training data. Finally, I tested the model by giving a new input [27, 2, 40000] and predicted whether the user will purchase or not.